

Flight Delay Analysis Project Report

1. Project Overview & Pipeline Design

Concept

This project processes and analyzes US domestic flight delays and cancellations (2009–2018). It utilizes Apache Hadoop (HDFS for storage, YARN for cluster resource management) and PySpark for distributed data processing. The final results are served via an interactive Streamlit web application.

Batch Precompute Pattern

The data pipeline follows a batch precompute pattern to ensure production-grade performance:

- **Download:** Raw text CSV files are downloaded from Kaggle directly to the local disk.
- **Ingest:** A PySpark job loads the CSV files, enforces structural data types, and converts them into highly compressed, columnar Parquet files stored on HDFS.
- **Aggregate:** A second PySpark job processes the entire Parquet dataset on HDFS, executes heavy analytical queries, and outputs compact, aggregated tables.
- **Export:** The precomputed aggregates are exported back to local Parquet files.

Runtime Performance

When a user browses the web application, the system does not invoke Hadoop or Spark in the background. Every UI interaction reads only the lightweight, precomputed data, allowing the interface to respond instantly.

2. Cluster Architecture & Environment

Master Container

The cluster runs inside isolated Docker containers on a single host machine. The master node (`hadoop-master`) hosts all central management daemons: HDFS NameNode, Secondary NameNode, and YARN ResourceManager. It also runs a local HDFS DataNode, a YARN NodeManager, and the Spark Driver process that orchestrates the execution of the application.

Slave Container

The worker infrastructure is split into separate containers to simulate network distribution:

- `slave1-dn` runs the HDFS DataNode daemon to contribute distributed storage capacity.
- `slave1-nm` runs the YARN NodeManager daemon, which physically launches Spark Executor containers when resources are allocated.

3. Technical Implementation

Data Ingestion Job

The ingestion phase is orchestrated by `ingest_all.sh`. It sequentially uploads each CSV file to HDFS using `hdfs dfs -put`, submits the PySpark job, and purges the temporary files to manage disk space.

Storage Efficiency

Data is written using Hive-style partitioning (`year=YYYY`) and an overwrite mode to guarantee pipeline idempotency. Due to Snappy compression and the column-oriented layout of Parquet, the dataset size is reduced from 7.2 GB to 1.3 GB on HDFS.

4. Distributed Analytics & Aggregation

Compute Aggregates Job

The core analytical workload is handled by `compute_aggregates.py`. It is submitted to YARN via `spark-submit` using conservative resource constraints: 1 Executor, 2 CPU cores, 1536 MB of executor memory, and 768 MB of driver memory. This allocation maintains cluster stability on a 16 GB host machine.

Target Aggregates

The job processes the 1.3 GB Parquet dataset and generates 8 independent business tables.

5. Web Application UI (Streamlit)

Dashboard Views

The interface features 7 distinct analytical views:

- **Airline Delays:** Ranks carriers by average arrival delay and delay percentages, tracking historical performance trends.
- **Flight Cancellations:** Analyzes cancellation rates across months and days of the week alongside cause distributions.
- **Airport Taxi-Out Times:** Identifies airports with long taxiing durations, featuring a minimum flight volume threshold to filter out low-traffic locations.
- **Delay Causes by Airport:** Visualizes the breakdown of delay minutes for any chosen airport.
- **Short vs Long Haul Flights:** Compares how carriers perform on short, medium, and long-haul routes.
- **Busiest Routes:** Displays high-volume flight corridors with automated color-coding to highlight poor punctuality.

Recommendation Engine

The final view is a passenger recommendation engine. For a chosen origin-destination pair, it evaluates carriers using a risk score formula.

6. Troubleshooting & Resolutions

Slave Executor Classpath Failures

- **Symptom:** Submitting Spark jobs to YARN resulted in immediate container failures on the slave node, accompanied by a `NoClassDefFoundError` for core YARN utility classes.
- **Root Cause:** The master container ran an image containing Hadoop 3.3.4, while the slave container was initialized using the official `apache/hadoop:3` image (Hadoop 3.3.6). The mismatch in directory layouts and JAR library versions prevented the slave from deserializing task objects sent by the master.
- **Resolution:** Both the master and slave were reconfigured to boot from the identical `adamgput/hadoop-pyspark-put:1.0` image. By overriding the container entrypoints, the same image was forced to act as the manager on the master and as pure storage/execution daemons (`hdfs datanode` and `yarn nodemanager`) on the slave.

Unhealthy Cluster Nodes via Disk Thresholds

- **Symptom:** During active development, the YARN ResourceManager UI flagged the NodeManagers as `UNHEALTHY` and rejected all subsequent job submissions.
- **Root Cause:** Docker build caches and dangling images accumulated on the host machine, pushing disk utilization past YARN's default safety threshold of 90%.
- **Resolution:** The host disk space was reclaimed by executing `docker builder prune -f`, which cleared 69 GB of data. Additionally, the cluster architecture was scaled down to a single slave node to reduce storage overhead while preserving the distributed nature of the cluster.